

Supplementary Material

Enabling Behavioral Diversity for Learning Locomotion in Humanoid Robots

Author Names Omitted for Anonymous Review. Paper-ID [716]

This document provides supplementary materials and extended analysis to support the claims and details of the approach *Discovery and Learning of Diverse Locomotion* (DLDL), presented in the main paper *Enabling Behavioral Diversity for Learning Locomotion in Humanoid Robots*. This supplement is organized as follows: Section I presents the full mathematical derivations for the losses used in the policy training. Furthermore, we include the mathematical derivation of the computational complexity of our DLDL training. Section II outlines the implementation specifics, including the simulation environment for evaluation, hyperparameter values used in the auxiliary policy learning, the details of evaluation metrics calculation. Finally, Section III includes additional qualitative results of DLDL, and the discussion of different baseline controller implementation designs.

I. MATHEMATICAL DERIVATIONS OF BOUNDS AND LOSSES

This section first lists the variables and mathematical notations in Table I used in our mathematical expressions and derivations, and then provides the mathematical derivations of the variational lower bounds and loss functions.

A. Derivation of the Variational Lower Bound of Mutual Information

We show the derivation of the variational lower bound of mutual information used to formulate $\mathcal{L}_I$, Eq. (3) in the main manuscript. We first derive the expected KL divergence between $p(\mathbf{z}|\mathbf{s}, \mathbf{a})$ and $p(\mathbf{z})$ by the following:

$$\max_{\mathbf{z}} I(\mathbf{z}; \mathbf{s}, \mathbf{a}) \quad (1)$$

$$= \max_{\mathbf{z}} D_{\text{KL}}(p(\mathbf{z}, \mathbf{s}, \mathbf{a}) \| p(\mathbf{z})p(\mathbf{s}, \mathbf{a})) \quad (2)$$

$$= \max_{\mathbf{z}} \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} \left[\log \frac{p(\mathbf{z}|\mathbf{s}, \mathbf{a})p(\mathbf{s}, \mathbf{a})}{p(\mathbf{s}, \mathbf{a})p(\mathbf{z})} \right] \quad (3)$$

$$= \max_{\mathbf{z}} \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} \left[\log \frac{p(\mathbf{z}|\mathbf{s}, \mathbf{a})}{p(\mathbf{z})} \right] \quad (4)$$

$$= \max_{\mathbf{z}} \mathbb{E}_{p(\mathbf{s}, \mathbf{a})} \left[\mathbb{E}_{p(\mathbf{z}|\mathbf{s}, \mathbf{a})} \left[\log \frac{p(\mathbf{z}|\mathbf{s}, \mathbf{a})}{p(\mathbf{z})} \right] \right] \quad (5)$$

$$= \max_{\mathbf{z}} \mathbb{E}_{p(\mathbf{s}, \mathbf{a})} [D_{\text{KL}}(p(\mathbf{z}|\mathbf{s}, \mathbf{a}) \| p(\mathbf{z}))] \quad (6)$$

In this derivation, we apply the chain rule of probability to Eq. (3) and Eq. (4) and derive the expectation in Eq. (5). We finally represent Eq. (5) as the KL divergence in Eq. (6).

Since the latent variable distribution $p(\mathbf{z})$ is intractable, we introduce the posterior $q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})$ to compute the variational

lower bound of the expected KL divergence Eq. (6). Our derivation is shown as the following:

$$\mathbb{E}_{p(\mathbf{s}, \mathbf{a})} [D_{\text{KL}}(p(\mathbf{z}|\mathbf{s}, \mathbf{a}) \| p(\mathbf{z}))] \quad (7)$$

$$= \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} [\log p(\mathbf{z}|\mathbf{s}, \mathbf{a}) - \log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) + \log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) - \log p(\mathbf{z})] \quad (8)$$

$$= \mathbb{E}_{p(\mathbf{s}, \mathbf{a})} [D_{\text{KL}}(p(\mathbf{z}|\mathbf{s}, \mathbf{a}) \| q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}))] + \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} [D_{\text{KL}}(q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \| p(\mathbf{z}))] \quad (9)$$

$$\geq \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} [\log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})] - \mathbb{E}_{p(\mathbf{z})} [\log p(\mathbf{z})] \quad (10)$$

$$= \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} [\log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})] + H(\mathbf{z}) \quad (11)$$

$$\geq \mathbb{E}_{p(\mathbf{s}, \mathbf{a}, \mathbf{z})} [\log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})] \quad (12)$$

After the introduction of $q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})$ in Eq. (8), the expression is then regrouped in Eq. (9) to formulate two KL divergence terms. By dropping the first term, a lower bound is established in Eq. (10). Then, We simplify this expression by identifying the entropy of the prior $H(\mathbf{z})$ in Eq. (11). Finally, since $H(\mathbf{z})$ is constant with respect to the learning process, it is removed to yield the final, tractable lower bound in Eq. (12).

B. Derivation of the Loss for Stable Posterior Learning

We derive the loss $\mathcal{L}_q$, Eq. (5) in the main manuscript, for learning posterior by following the method introduced by AWAC [1] and DiveOff [2]. We start from introducing a normalization constraint to the objective for stable posterior learning, which ensures that the encoder network can model a valid posterior distribution for different latent variables instead of modeling a biased posterior for only a single latent variable. We formulate the problem as:

$$\max_{q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})} \mathbb{E}_{\mathbf{z} \sim q_{\Theta}(\cdot|\mathbf{s}, \mathbf{a})} [A(\mathbf{s}, \mathbf{a}, \mathbf{z})] \quad (13)$$

$$\text{s.t. } D_{\text{KL}}(q_{\Theta}(\cdot|\mathbf{s}, \mathbf{a}) \| q_{\text{old}}(\cdot|\mathbf{s}, \mathbf{a})) \leq \epsilon_q \quad (14)$$

$$\int q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) d\mathbf{z} = 1 \quad (15)$$

Then, we formulate the Lagrangian of Eq. (13) as:

$$\begin{aligned} L(q_{\Theta}, \alpha_q, \mu_q) = & \mathbb{E}_{\mathbf{z} \sim q_{\Theta}(\cdot|\mathbf{s}, \mathbf{a})} [A(\mathbf{s}, \mathbf{a}, \mathbf{z})] \\ & - \alpha_q (D_{\text{KL}}(q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \| q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}))) \\ & + \mu_q \left(\int q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) d\mathbf{z} - 1 \right) \end{aligned} \quad (16)$$

Then, we calculate the derivative of Eq. (16) with respect to $q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})$ and present it as:

$$\frac{\partial L(q_{\Theta}, \alpha_q, \mu_q)}{\partial q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})} = A(\mathbf{s}, \mathbf{a}, \mathbf{z}) - \alpha_q \left(\log \frac{q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})}{q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a})} + 1 \right) + \mu_q \quad (17)$$

We set the derivative Eq. (17) to zero and solve it to find the extrema of this function. We present the solution as:

$$q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) = q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{A(\mathbf{s}, \mathbf{a}, \mathbf{z})}{\alpha_q}\right) \exp\left(\frac{\mu_q}{\alpha_q} - 1\right) \quad (18)$$

$$\propto q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \quad (19)$$

We ignore the term $\exp(\frac{\mu_q}{\alpha_q} - 1)$ in Eq. (18) since it is constant with respect to $\mathbf{z}$, yielding the final solution in Eq. (19). We minimize the KL divergence between the posterior $q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})$ and the posterior proportional to the extrema point $q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)$, yielding the following equations:

$$D_{\text{KL}}(q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \| q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})) \quad (20)$$

$$\begin{aligned} &= \int q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \\ &\quad \log\left(\frac{q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)}{q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})}\right) d\mathbf{z} \quad (21) \\ &= \int q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \log q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) d\mathbf{z} \\ &\quad + \frac{1}{\alpha_q} \int q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) A(\mathbf{s}, \mathbf{a}, \mathbf{z}) d\mathbf{z} \\ &\quad - \int q_{\text{old}}(\mathbf{z}|\mathbf{s}, \mathbf{a}) \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a}) d\mathbf{z} \quad (22) \end{aligned}$$

Since the first and second term in Eq. (22) is independent with the posterior $q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})$, we can ignore them and formulate the loss for learning posterior with the third term in Eq. (22):

$$\max_{q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mathbf{D}} \mathbb{E}_{\mathbf{z} \sim q_{\text{old}}(\cdot|\mathbf{s}, \mathbf{a})} [W_q(\mathbf{s}, \mathbf{a}, \mathbf{z}) \log q_{\Theta}(\mathbf{z}|\mathbf{s}, \mathbf{a})] \quad (23)$$

We let $W_q(\mathbf{s}, \mathbf{a}, \mathbf{z}) = \exp\left(\frac{1}{\alpha_q} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)$, yielding the loss for stable posterior learning.

C. Derivation of the Loss for Behavioral-Diverse Policy Learning

The loss $\mathcal{L}_{\pi}$, Eq. (8) in the main manuscript, for learning the policy $\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$ is derived using the same procedure in I-B. We begin with introducing a normalization constraint, ensuring the learned policy network can model the distributions of different actions, to the objective for policy learning and formulate the following problem:

$$\max_{\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})} \mathbb{E}_{\mathbf{a} \sim \pi(\cdot|\mathbf{s}, \mathbf{z})} [A(\mathbf{s}, \mathbf{a}, \mathbf{z})] \quad (24)$$

$$\text{s.t. } D_{\text{KL}}(\pi(\cdot|\mathbf{s}, \mathbf{z}) \| \beta(\cdot|\mathbf{s}, \mathbf{z})) \leq \epsilon_{\pi} \quad (25)$$

$$\int \pi(\mathbf{a}|\mathbf{s}, \mathbf{z}) d\mathbf{a} = 1 \quad (26)$$

TABLE I
MATHEMATICAL NOTATIONS.

Variable	Definition
$\mathbf{s}$	State
$\mathbf{a}$	Action
$\mathbf{z}$	Latent variable
$\mathbf{g}_t^{(i)}$	Lookahead reference poses for skill i
$\hat{\mathbf{q}}_t^{(i)}$	Reference motion for skill i
k	Number of locomotion skills
$\mathbf{D}$	Offline dataset
$\mathcal{L}$	Loss
Ω	Parameters of the baseline policy network
Ψ	Parameters of the auxiliary policy network
Θ	Parameters of the inference network
r	Reward
ϵ	KL divergence constraints
λ	Coefficient for balancing mutual information loss
α	Coefficients balancing advantage weighting
μ	Lagrangian multipliers
Policy	Definition
$\beta(\mathbf{a} \mathbf{s}, \mathbf{z})$	Underlying behavior policy
$\pi_{\Omega}^{(i)}(\mathbf{a} \mathbf{s}, \mathbf{g}_t^{(i)})$	Baseline policy for skill i
$\pi_{\Psi}(\mathbf{a} \mathbf{s}, \mathbf{z})$	Auxiliary policy
$\tilde{\pi}_{\Psi}(\mathbf{a} \mathbf{s}, \mathbf{z})$	Preliminary auxiliary policy (Round 1)
$\pi_f(\mathbf{a} \mathbf{s}, \mathbf{z})$	Final composed policy
$\tilde{\pi}_f(\mathbf{a} \mathbf{s}, \mathbf{z})$	Preliminary composed policy (Round 1)
$\mu_{\Omega}, \Sigma_{\Omega}$	Mean, covariance of the baseline policy
$\mu_{\Psi}(\mathbf{s}, \mathbf{z}), \Sigma_{\Psi}(\mathbf{s}, \mathbf{z})$	Mean, covariance of the auxiliary policy
$\mu_f(\mathbf{s}, \mathbf{z}), \Sigma_f(\mathbf{s}, \mathbf{z})$	Mean, covariance of the composed policy
Function	Definition
$A(\mathbf{s}, \mathbf{a}, \mathbf{z})$	Advantage function
$V(\mathbf{s}, \mathbf{z})$	State-value function
$Q_1(\mathbf{s}, \mathbf{a}, \mathbf{z}), Q_2(\mathbf{s}, \mathbf{a}, \mathbf{z})$	Critic networks
$W_q(\mathbf{s}, \mathbf{a}, \mathbf{z}), W_{\pi}(\mathbf{s}, \mathbf{a}, \mathbf{z})$	Advantage-weighting functions
$q_{\Theta}(\mathbf{z} \mathbf{s}, \mathbf{a})$	Encoder
$l_{\Theta}(\mathbf{s}, \mathbf{a} \mathbf{z})$	Decoder

Then, we formulate the Lagrangian of Eq. (24) as:

$$\begin{aligned} L(\pi, \alpha_{\pi}, \mu_{\pi}) &= \mathbb{E}_{\mathbf{a} \sim \pi(\cdot|\mathbf{s}, \mathbf{z})} [A(\mathbf{s}, \mathbf{a}, \mathbf{z})] \\ &\quad - \alpha_{\pi} (D_{\text{KL}}(\pi(\mathbf{a}|\mathbf{s}, \mathbf{z}) \| \beta(\mathbf{a}|\mathbf{s}, \mathbf{z})) - \epsilon_{\pi}) \quad (27) \\ &\quad + \mu_{\pi} \left(\int \pi(\mathbf{a}|\mathbf{s}, \mathbf{z}) d\mathbf{a} - 1 \right) \end{aligned}$$

Next, we calculate the functional derivative of Eq. (27) with respect to $\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$:

$$\frac{\partial L(\pi, \alpha_{\pi}, \mu_{\pi})}{\partial \pi(\mathbf{a}|\mathbf{s}, \mathbf{z})} = A(\mathbf{s}, \mathbf{a}, \mathbf{z}) - \alpha_{\pi} \left(\log \frac{\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})}{\beta(\mathbf{a}|\mathbf{s}, \mathbf{z})} + 1 \right) + \mu_{\pi} \quad (28)$$

We set the derivative to zero and find the extrema with respect to $\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$, yielding the solution:

$$\pi(\mathbf{a}|\mathbf{s}, \mathbf{z}) = \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{A(\mathbf{s}, \mathbf{a}, \mathbf{z})}{\alpha_{\pi}}\right) \exp\left(\frac{\mu_{\pi}}{\alpha_{\pi}} - 1\right) \quad (29)$$

$$\propto \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_{\pi}} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \quad (30)$$

We ignore the term $\exp\left(\frac{\mu_{\pi}}{\alpha_{\pi}} - 1\right)$ in Eq. (29) as it is constant. Then, we formulate an objective to minimize the KL divergence between our learnable policy $\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$ and

the optimal solution $\beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)$, yielding the following equations:

$$D_{\text{KL}}(\beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \parallel \pi(\mathbf{a}|\mathbf{s}, \mathbf{z})) \quad (31)$$

$$\begin{aligned} &= \int \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \\ &\quad \log\left(\frac{\beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)}{\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})}\right) d\mathbf{a} \quad (32) \\ &= \int \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \log \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) d\mathbf{a} \\ &\quad + \frac{1}{\alpha_\pi} \int \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) A(\mathbf{s}, \mathbf{a}, \mathbf{z}) d\mathbf{a} \\ &\quad - \int \beta(\mathbf{a}|\mathbf{s}, \mathbf{z}) \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right) \log \pi(\mathbf{a}|\mathbf{s}, \mathbf{z}) d\mathbf{a} \quad (33) \end{aligned}$$

The first and second term in Eq. (33) are independent with the policy $\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$, so we ignore them and take the third term to construct the loss. Let $W_\pi(\mathbf{s}, \mathbf{a}, \mathbf{z}) = \exp\left(\frac{1}{\alpha_\pi} A(\mathbf{s}, \mathbf{a}, \mathbf{z})\right)$, we formulate the loss as maximizing the following objective:

$$\max_{\pi(\mathbf{a}|\mathbf{s}, \mathbf{z})} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mathbf{D}} \mathbb{E}_{\mathbf{z} \sim q_\Theta(\cdot|\mathbf{s}, \mathbf{a})} [W_\pi(\mathbf{s}, \mathbf{a}, \mathbf{z}) \log \pi(\mathbf{a}|\mathbf{s}, \mathbf{z})] \quad (34)$$

D. Complexity Analysis

We analyze the time complexity of the DLDL training and deployment procedure to assess the scalability and efficiency of our approach. To compute the time complexity, we calculate the number of basic computational operations required for training, such as multiplication and addition. We assume the baseline policy network, the auxiliary policy network, the critic networks, and the inference networks are configured with the same structure, such as the number of hidden layers, and the dimensionality of the input, output, and hidden layers. We define d^{in} as the dimensionality of the input layer, d^{out} as the dimensionality of the output layer, and h as the dimensionality of a hidden layer, assuming that $d^{\text{in}}, d^{\text{out}} \leq h$. We also define B as the size of a data batch, and c as a constant number representing the cost of computational operations in neural networks.

Baseline Policy Training. For each of the k locomotion skills, we train a baseline policy $\pi_\Omega^{(i)}$ via motion imitation. The computational cost per training iteration for a single baseline policy is $c \cdot B \cdot [(d_\Omega^{\text{in}} + 1)h + (h + 1)d_\Omega^{\text{out}} + h^2 + h] = \mathcal{O}(Bh^2)$. Let T_Ω be the number of training iterations per baseline policy; then the computational complexity of training all k baseline policies is $\mathcal{O}(kT_\Omega Bh^2)$.

Auxiliary Policy Learning. Before the main training loop, we initialize the encoder q_Θ and decoder l_Θ by training them with the offline dataset. We derive the complexity as $\mathcal{O}(Bh^2)$ per iteration. Given T_{init} initialization iterations, the computational complexity of this step is $\mathcal{O}(T_{\text{init}} Bh^2)$.

The main training loop within the auxiliary policy learning procedure consists of the training of the critic networks Q_1, Q_2 , the inference networks q_Θ, l_Θ , and the auxiliary

policy network π_Ψ . Each has a per-iteration training cost of $\mathcal{O}(Bh^2)$. Let T_Ψ be the number of training iterations for this loop, the computational complexity of one call to the auxiliary policy learning procedure is $\mathcal{O}(T_{\text{init}} Bh^2 + T_\Psi Bh^2)$. Since this procedure is invoked once per round of dataset construction, the overall computational complexity of auxiliary policy learning is $\mathcal{O}(T_{\text{init}} Bh^2 + T_\Psi Bh^2)$.

Policy Composition. At both dataset construction and deployment, composing the baseline policy and the auxiliary policy via the closed-form product of experts requires only elementwise operations on the covariance matrices and a weighted sum of the two means. We therefore derive the computation cost as $\mathcal{O}(d_a)$, where d_a is the dimensionality of the action space.

Overall Complexity. Combining the above, the overall computational complexity of training DLDL is $\mathcal{O}(kT_\Omega Bh^2 + T_{\text{init}} Bh^2 + T_\Psi Bh^2)$. At deployment, DLDL's policy execution depends on a single forward inference from the baseline policy network and the auxiliary policy network, followed by the closed-form composition. The computational cost is bounded by $\mathcal{O}(h^2)$.

II. ADDITIONAL IMPLEMENTATION AND EVALUATION DETAILS

A. Simulation Environment Design

To evaluate humanoid locomotion strategies, we developed a simulated warehouse environment that captures the spatial complexity and constraints commonly encountered in real-world industrial settings. Warehouses present a variety of confined spaces, obstacles, and operational constraints that necessitate diverse locomotion behaviors from humanoid robots.

The environment is structured into three distinct zones, each specifically designed to assess a targeted locomotion capability:

- **Low-Clearance Zone:** This area features a partially open roller shutter that simulates a physical height constraint. To navigate through this space, the humanoid robot must execute a *sneaking* behavior – lowering its body posture to avoid contact with the overhead barrier. This task tests the robot's ability to modulate its body height dynamically while maintaining balance and forward motion.
- **Narrow Passage Zone:** Here, a tight corridor is formed between a conveyor belt and stacks of cargo, emulating the kind of cluttered and restrictive spaces often found in operational warehouses. Standard forward walking is impractical in this zone; instead, the humanoid robot is required to perform *side-stepping* motions. This scenario evaluates the robot's lateral maneuverability and its ability to maintain stable locomotion in constrained environments.
- **Open Navigation Zone:** This area provides an unobstructed floor space with a cargo box placed arbitrarily to serve as an obstacle. The humanoid robot is tasked with performing *forward walking* while incorporating

basic obstacle avoidance strategies. This setup serves as a baseline to validate the robot’s fundamental walking capabilities and reactive motion planning in the presence of minor environmental interference.

Together, these zones form a comprehensive benchmark that challenges a humanoid robot to switch between distinct locomotion strategies in response to varied spatial constraints, closely mirroring the demands of real-world deployment scenarios in logistics and industrial automation.

We selected the Genesis simulator as our evaluation platform due to its suitability for research in humanoid locomotion. Genesis offers a high-fidelity physics engine capable of accurately modeling the complex, contact-rich dynamics inherent to humanoid motion, such as ground contact, balance control, and joint compliance. This level of physical realism is critical for developing and evaluating robust locomotion behaviors. In addition, Genesis provides high-throughput simulation performance, which significantly accelerates both training and evaluation phases. This is particularly advantageous when using data-intensive methods such as reinforcement learning or large-scale behavior cloning, where simulation speed can be a major bottleneck. Finally, the simulator offers flexible support for environment customization, allowing us to design and implement the diverse warehouse scenarios required for our experiments. This feature is essential for validating our method’s ability to generalize across a range of locomotion tasks and environmental constraints.

B. Hyperparameters for Auxiliary Policy Learning

We present the hyperparameters and network structures used for auxiliary policy learning in Table II. The baseline policy is trained via motion imitation following the existing ProtoMotions implementation [3], [4]; we refer readers to this implementation for the corresponding baseline policy training hyperparameters.

C. Quality-Diversity(QD) Score Calculation

We followed the previous works [5]–[8] to calculate the Quality-Diversity(QD) score. We construct an archive to store the number of survival steps of each diverse locomotion behavior. We define an archive as a grid space with n dimensions. We define $[l_i, u_i], i \in [1, n]$ as the lower and upper bound of the space on i^{th} dimension of the grid space. We uniformly discretize the space on i^{th} dimension into c grids. Given one locomotion evaluation, we take the number of survival steps τ as a score. Then, we compute an index tuple $(b_1, \dots, b_n), b_i \in [l_i, u_i]$ that characterizes the behavior in this evaluation. This continuous vector is then mapped to a discrete grid in the archive. The score in that grid is updated with τ if the grid is empty or τ is greater than the score currently stored there. Let $\tau_{b_1, \dots, b_n}$ be the score stored in the grid indexed by $(b_1, \dots, b_n)$, we then compute the QD score measuring how well a policy can simultaneously generate effective locomotion and diverse behaviors by:

$$QD(\pi) = \sum_{b_1=1}^c \dots \sum_{b_n=1}^c \tau_{b_1, \dots, b_n} \quad (35)$$

TABLE II
HYPERPARAMETERS AND NETWORK STRUCTURES USED FOR AUXILIARY
POLICY LEARNING.

Hyperparameter	Value
Learning rate for policy network	3e-4
Learning rate for critic network	3e-4
Learning rate for posterior distribution	3e-4
Learning rate for likelihood	3e-4
Learning rate for mutual information loss $\mathcal{L}_I$	6e-5
Discount factor	0.99
Mini-batch size	256
Update rate for policy network parameters	5e-3
Update rate for critic network parameters	5e-3
Update frequency of policy network parameters	2
Balancing coefficient α_π	1/3
Balancing coefficient α_q	1.0
Balancing coefficient for mutual information λ	2.0
Dimensionality of hidden layers in policy network	1024
Dimensionality of hidden layers in critic network	1024
Dimensionality of encoder network	1024
Dimensionality of decoder network	1024
Dimensionality of latent variable	16
Number of hidden layers in policy network	2
Number of hidden layers in critic network	2
Number of hidden layers in encoder network	2
Number of hidden layers in decoder network	2
Activation function of policy network	ReLU
Activation function of critic network	ReLU
Activation function of encoder network	ReLU
Activation function of decoder network	ReLU

where a higher c indicates the higher archive’s granularity, allowing more evaluations with subtle varied behaviors are mapped to distinct grids.

In our experiments, we use a 3-dimensional vector where each dimension represents the average linear velocity along a cardinal axis (x, y, z) . The index tuple is therefore the average velocity vector (v_x, v_y, v_z) , with each component bounded with $[-0.5, 0.5]$.

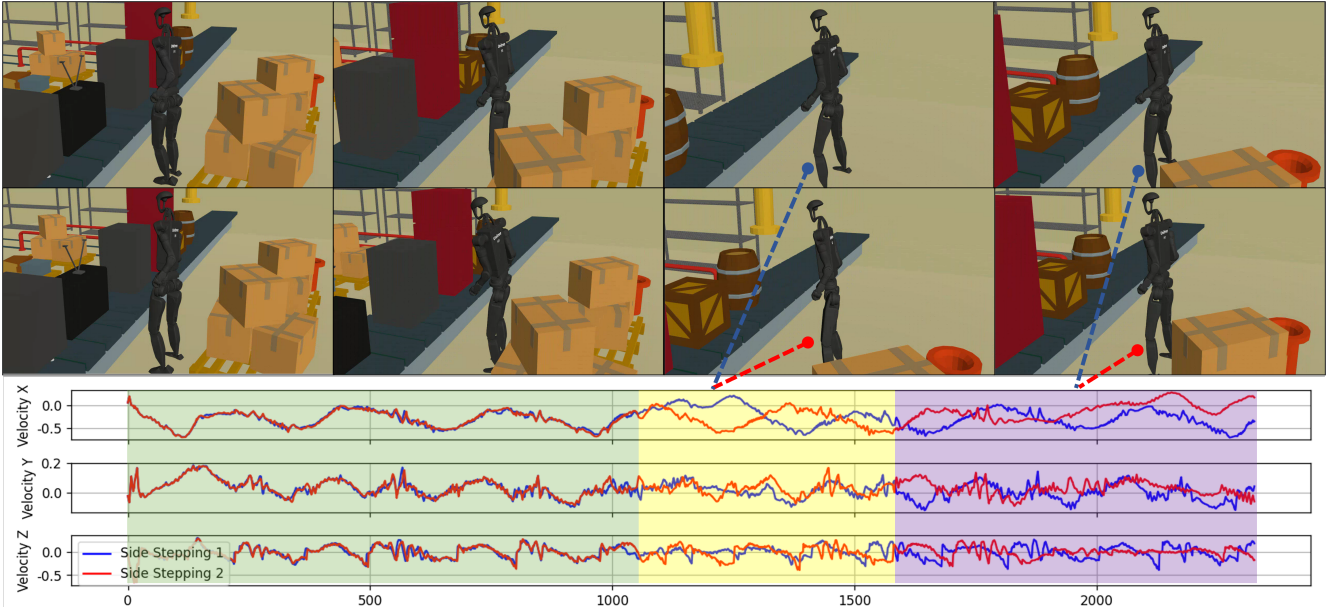
III. ADDITIONAL EXPERIMENTAL RESULTS

A. Additional Qualitative Results on Diverse Humanoid Locomotion

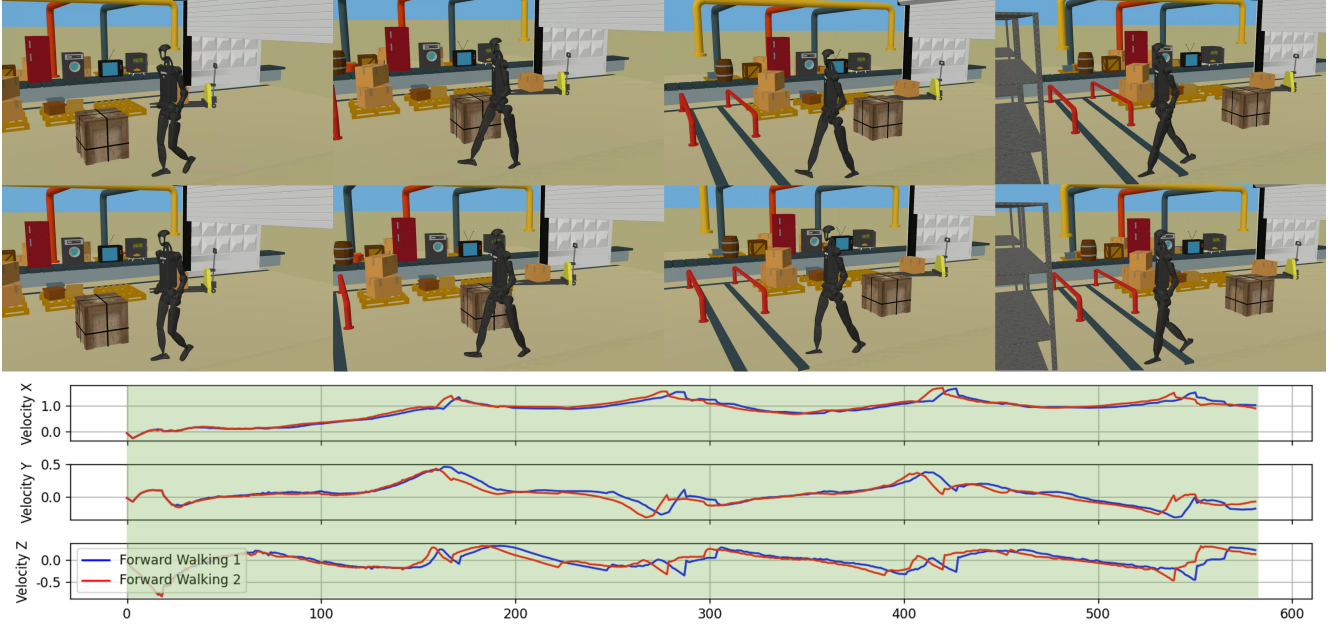
We perform further experiments using the Unitree H1-2 humanoid robot in Genesis simulations to demonstrate the diverse locomotion behaviors in scenarios of side-stepping and forward walking.

We illustrate a H1-2 humanoid performing side stepping in Fig. 1(a). Two humanoids start from the same initial position and each is initialized with a different latent variable. Their velocity profiles are nearly identical at first, but after step 1100 their behaviors begin to diverge. The first robot pauses in front of the conveyor belt while the second continues moving. After step 1300, the first robot resumes its motion, and the behavioral differences become more pronounced after step 1600. Ultimately, the first robot reaches the end of the conveyor and reverses direction, whereas the second remains stationary for the rest of the evaluation.

In addition, we present the qualitative results demonstrating H1-2 forward walking locomotion in Fig. 1(b). In this experiment, although the two H1-2 humanoids are conditioned



(a) The H1-2 humanoid robot learns diverse **side-stepping** behaviors to navigate a narrow passage between a conveyor belt and adjacent cargo piles.



(b) Diverse **forward-walking** behaviors learned by the H1-2 humanoid robot for locomotion in open areas.

Fig. 1. Additional results obtained by DLDL demonstrate that a H1-2 humanoid robot can learn diverse locomotion behaviors to navigate warehouse environments under varying constraints.

on different latent variables to encourage diversity, they performed the locomotion with nearly identical gaits. We hypothesize that this observed similarity is a result of the short execution time. Since the behavioral divergence driven by the latent variables accumulates gradually, a longer evaluation period is likely required for distinct walking styles to become apparent.

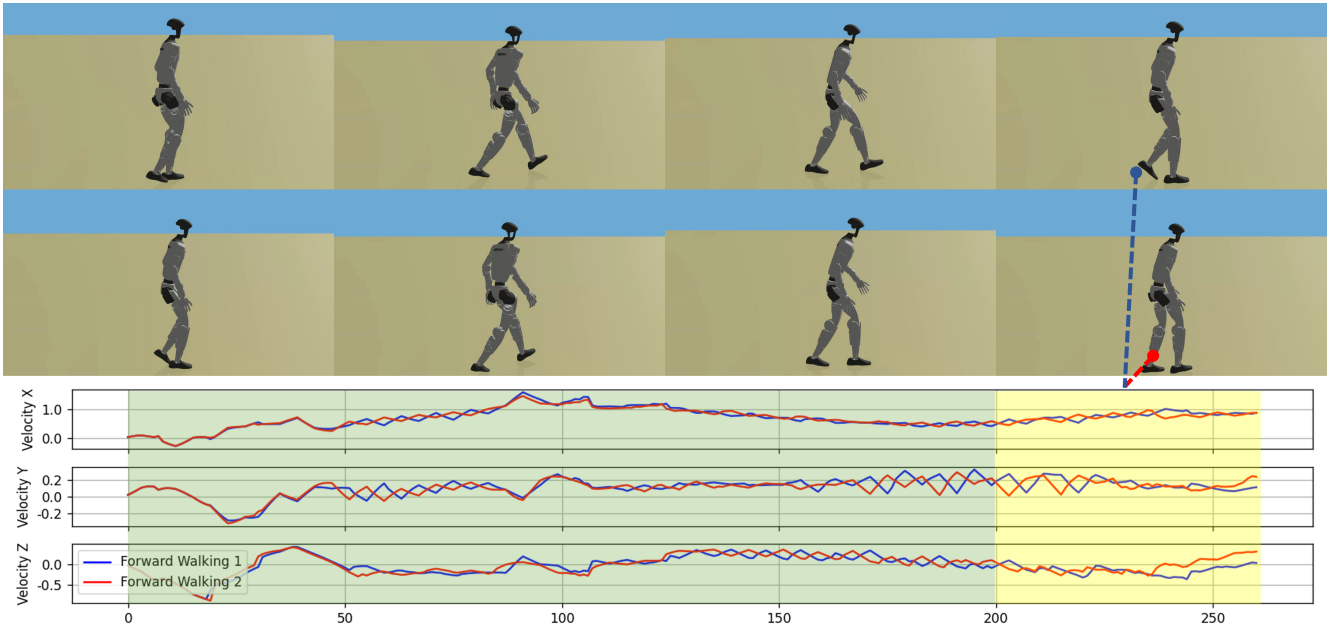
We also include the qualitative results of G1 robot performing sneaking and forward walking in Fig. 2. These results mirror the qualitative results observed on the H1-2 robot. The G1 robot generated clearly diverse behaviors for the

sneaking task, while the behavioral divergence during the *forward walking* task was minimal due to the insufficient time for diversity emergence.

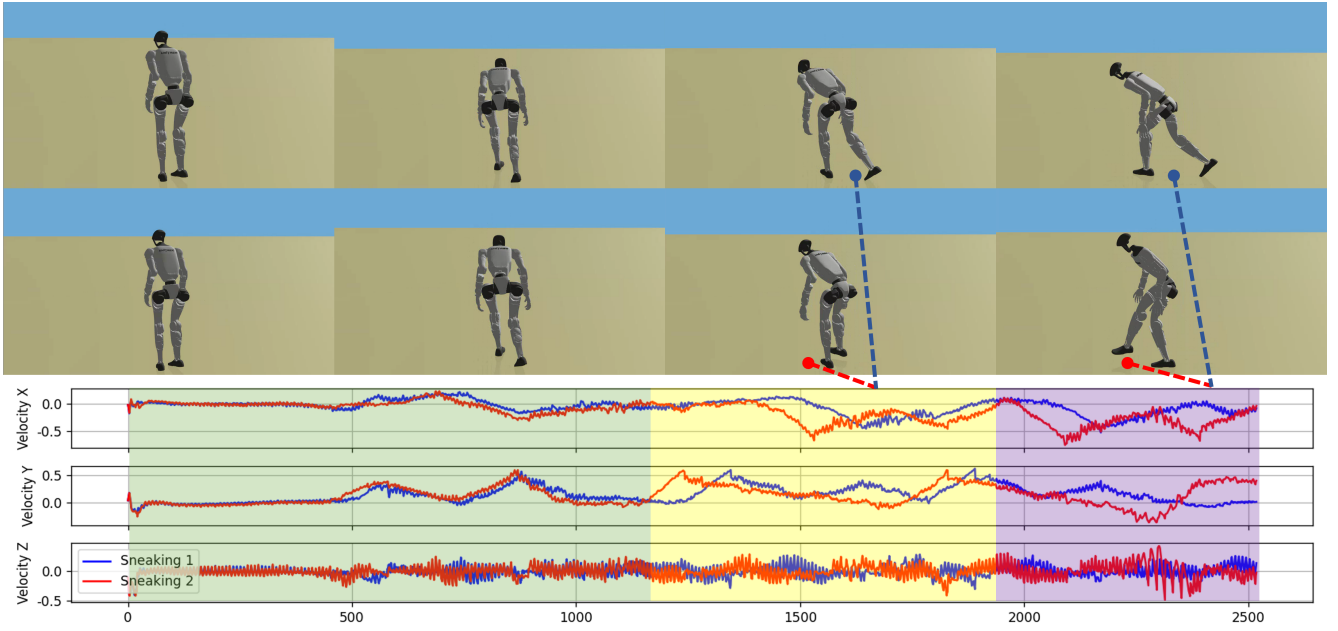
B. Discussion of Baseline Controller Covariance

Let the covariance of the baseline policy be $\Sigma_{\Omega} = \exp(x) \cdot \mathbf{I}$, as defined in the main manuscript, we provide the effect on QD scores with the difference value settings of x , while other hyperparameters remain unchanged. We present the results in Table III.

As we change the value of x and collect 30 evaluations



(a) Diverse **forward walking** behaviors learned by the G1 Humanoid in open areas.



(b) Diverse **sneaking** behaviors performed by the G1 Humanoid with two different step sizes.

Fig. 2. Qualitative results obtained by DLDL demonstrate that Unitree G1 humanoid can generate diverse locomotion behaviors.

TABLE III
OVERALL QD SCORE OF DLDL WITH DIFFERENT BASELINE CONTROLLER COVARIANCE SETTINGS.

Value of x	Forward Walking			Side Stepping			Sneaking		
	$c = 10$	$c = 25$	$c = 50$	$c = 10$	$c = 25$	$c = 50$	$c = 10$	$c = 25$	$c = 50$
$x = -1.2$	513	1595	1679	364	443	845	257	379	994
$x = -2.4$	421	580	901	4512	7501	11645	3584	4125	9391
$x = -3.6$	215	215	430	1878	3011	3011	1274	2548	2548
$x = -4.8$	215	215	430	1133	2266	3011	1274	2548	2548

under this setting, the overall QD scores change accordingly. We can observe that, our DLDL approach achieves the highest scores in *forward walking* task with the largest value $x = -1.2$ while it performs the worst in the other two tasks. With $x = -2.4$, DLDL achieves better scores compared to the ones with settings $x = -3.6$ and $x = -4.8$ in *forward walking* task, and outperforms all others in *side stepping* and *sneaking* tasks. When we set $x = -3.6$ and $x = -4.8$, DLDL's scores become lower. We reason that, as the value of x becomes too small, the confidence we assign to the baseline policy is too high. Such assignment can control the robot to perform more effective locomotion without falling, but greatly reduces the behavioral diversity and therefore leads lower QD scores. On the other hand, assigning too large value to x can achieve better behavioral diversity but hinder the effectiveness of locomotion. With $x = -1.2$, DLDL's output can not generate effective locomotion in *side stepping* and *sneaking* so it achieves lower QD scores. We also observe that, in simpler *forward walking* task, our method can generate effective locomotion without sacrificing behavioral diversity. We therefore select $x = -2.4$ as our setting for the baseline policy during evaluation, as it can balance the effective locomotion control and diverse behavior generation.

REFERENCES

- [1] A. Nair, A. Gupta, M. Dalal, and S. Levine, "Awac: Accelerating online reinforcement learning with offline datasets," *arXiv preprint arXiv:2006.09359*, 2020.
- [2] T. Osa and T. Harada, "Discovering multiple solutions from a single task in offline reinforcement learning," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [3] C. Tessler*, Y. Jiang*, X. B. Peng, E. Coumans, Y. Shi, H. Zhang, D. Rempe, G. Chechik†, and S. Fidler†, "Protomotions3: An open-source framework for humanoid simulation and control," <https://github.com/NVlabs/ProtoMotions/>, 2025.
- [4] C. Tessler, Y. Guo, O. Nabati, G. Chechik, and X. B. Peng, "Masked-mimic: Unified physics-based character control through masked motion inpainting," *ACM Transactions On Graphics (TOG)*, vol. 43, no. 6, pp. 1–21, 2024.
- [5] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.
- [6] M. Flageat, B. Lim, L. Grillotti, M. Allard, S. C. Smith, and A. Cully, "Benchmarking quality-diversity algorithms on neuroevolution for reinforcement learning," *arXiv preprint arXiv:2211.02193*, 2022.
- [7] J. K. Pugh, L. B. Soros, and K. O. Stanley, "Quality diversity: A new frontier for evolutionary computation," *Frontiers in Robotics and AI*, vol. 3, p. 40, 2016.
- [8] D. Gravina, A. Khalifa, A. Liapis, J. Togelius, and G. N. Yannakakis, "Procedural content generation through quality diversity," in *IEEE Conference on Games*, 2019.